Decimation and interpolation with polyphasefilters

Josef Hoffmann

Contents

1 The decimation process **1**
1.1 Introduction 1
1.2 Decimation of band limited signals 1
1.2.1 Simulation of a decimation 3

2 Decimation with polyphase realization **6**
2.1 The polyphase realization of FIR filters 8
2.2 Decimation with filter in polyphase realization 12

3 The interpolation process **15**
3.1 Introduction 15
3.2 Interpolation of band limited lowpass signals 15
3.2.1 Simulation of an interpolation 19

4 Interpolation with polyphase realization **23**

5 Appendix **29**
5.1 Understanding the 'Unbuffer' function 29

1 The decimation process

1.1 Introduction

First the decimation and interpolation is shown in the classical form [2], and then these operations with polyphase filters are described [1]. The mathematical proofs are detailed in the sources cited. The results from the literature are mainly used here, which are then supplemented with simulations in Simulink and with MATLAB scripts.

1.2 Decimation of band limited signals

The decimation process is shown in Fig. 1a. The input signal is band limited with an FIR low-pass filter up to a highest frequency f_{max} and further decimated

with a down sampler. The signal $z[nT_s]$ is sampled with a sampling period T_s or sampling frequency f_s. In the case of a decimation with factor M, only every M-th sample is retained, as shown in Fig. 1b.

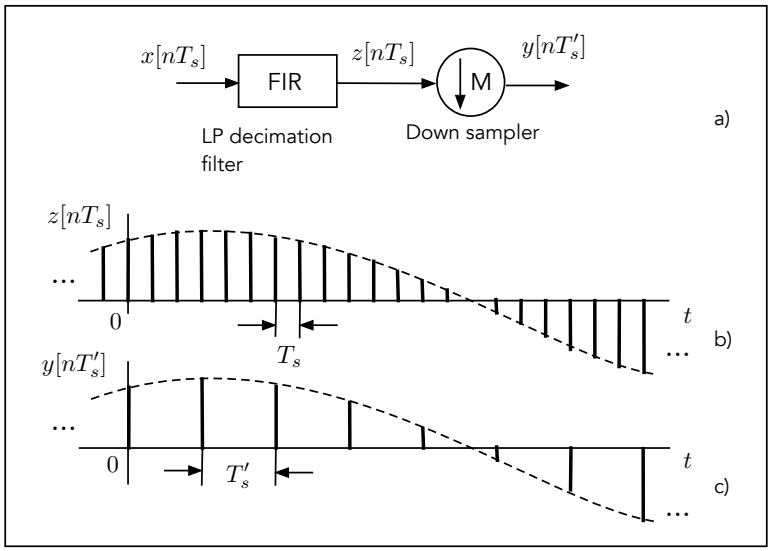


Fig. 1: Decimation process

As a result, the signal $y[nT'_s]$ now has a sampling period $T'_s = MT_s$ and a lower sampling frequency equal to $f'_s = f_s/M$. For the decimated signal to have the same information as the original useful signal, the decimation factor M must not exceed a certain value. In Fig.2a the spectrum of the input signal is shown. It consists of the useful or desired signal with a maximum bandwidth of up to f_{max} and of a perturbation. These can lead to distortions by aliasing after decimation.

With the help of the FIR low-pass filter, the perturbation and possibly also the part in the input signal that is not wanted are suppressed. The result is the signal $z[nT_s]$ with a spectrum as shown in Fig. 2b.
The downsampling of the signal $z[nT_s]$ then results in the spectrum from Fig. 2c [1]. So that no aliasing is taking place, the new sampling frequency must meet the following condition:

$$f'_s \geq 2f_{max} \quad \text{or} \quad \frac{f_s}{M} \geq 2f_{max} \tag{1}$$

From this follows the condition for M:

$$M \leq \left\langle \frac{f_s}{2f_{max}} \right\rangle \tag{2}$$

The $\langle \quad \rangle$ denotes the smallest whole value of this fraction. As an example for $f_{max} = 6$ kHz and $f_s = 100$ kHz, the ratio $f_s/(2f_{max}) = 8.333$ and thus

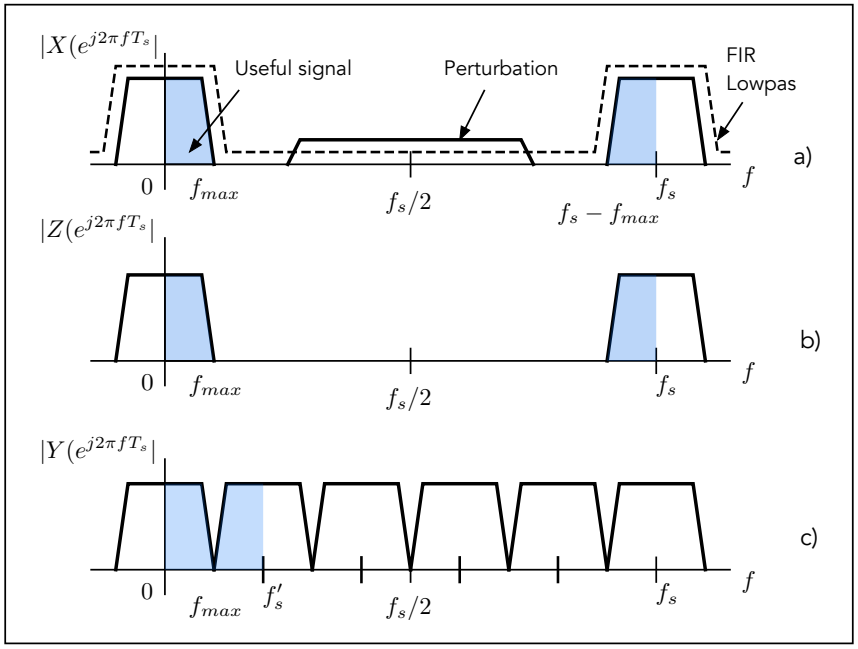


Fig. 2: Spectra of the decimation process

$M = 8$ is selected. The pass frequency of the FIR filter relative to f_s is equal to $f_{max}/f_s = f_{max}/(M2f_{max}) = 1/(2M)$.

In the MATLAB convention, the relative frequencies are defined relative to half the sampling frequency and therefore in this convention the relative pass frequency of the FIR filter is equal to $1/M$.

1.2.1 Simulation of a decimation

With the Simulink model `decim1.slx`, which is initialized and called from the script `decim_1.m`, a decimation is simulated. The model is shown in Fig. 3. The following instructions are used to initialize the model in the script:

```
% ------ Initialisations
fs = 100000;     Ts = 1/fs;   % Sample frequency and period
fmax = 6000;                  % Bandwidth of the useful signal
nord = 128;
hmax = fir1(nord, 2*fmax/fs); % Filter for the useful signal

f1 = 20e3;            % Starting frequency for the perturbance
nord1 = 128;
hper = fir1(nord1,f1*2/fs,'high'); % Filter for the perturbance
pert = 0.2;           % Gain for the perturbation

M = floor(fs/(2*fmax)),  % Decimation factor
```

```
nord2 = 2*128;              % Decimation Filter
%hdec = fir1(nord2, 1/M); % FIR decimation filter
hdec = fir1(nord2, 1.1/M); % FIR decimation filter
```

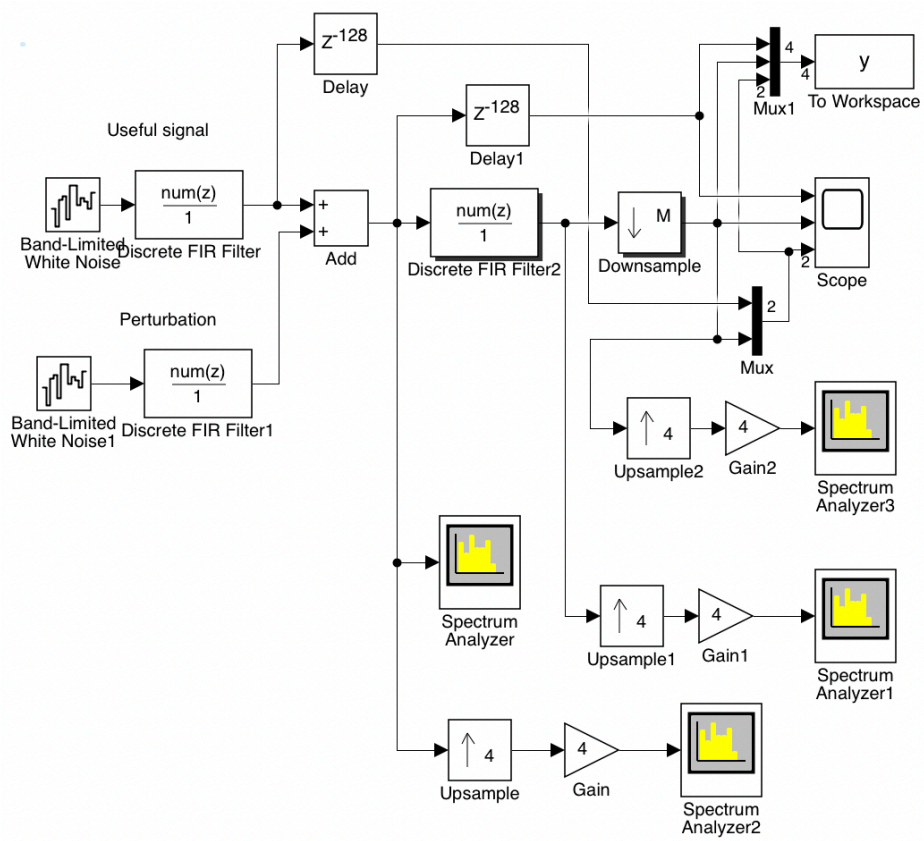


*Fig. 3: Simulink model of a decimation (*decim_1.m, decim1.slx)

With the blocks `Band-Limited White Noise` and `Discrete FIR Filter` a band limited random signal is generated as the useful signal. At a sampling frequency of `fs` =100 kHz, the filter with impulse response `hmax` generates the random signal with a bandwidth equal to `fmax` = 6 kHz.

With the blocks `Band-Limited White Noise1` and `Discrete FIR Filter1`, the perturbance is similarly generated as a random signal with components above the frequency `f1` = 20 kHz. The impulse response of this filter is `hper` and with the amplification factor `pert` one can choose the strength of the perturbance.

According to Eq. 2 the decimation factor M is calculated. The decimation filter of the impulse response `hdec` is implemented in the block `Discrete FIR Filter2`. There are two options in the script for calculating

`hdec` which will be explained in more detail later.

The downsampling is implemented in the block `Downsample`. The results of the simulation are shown with the `Scope` block and stored temporarily in the `To Workspace` sink.

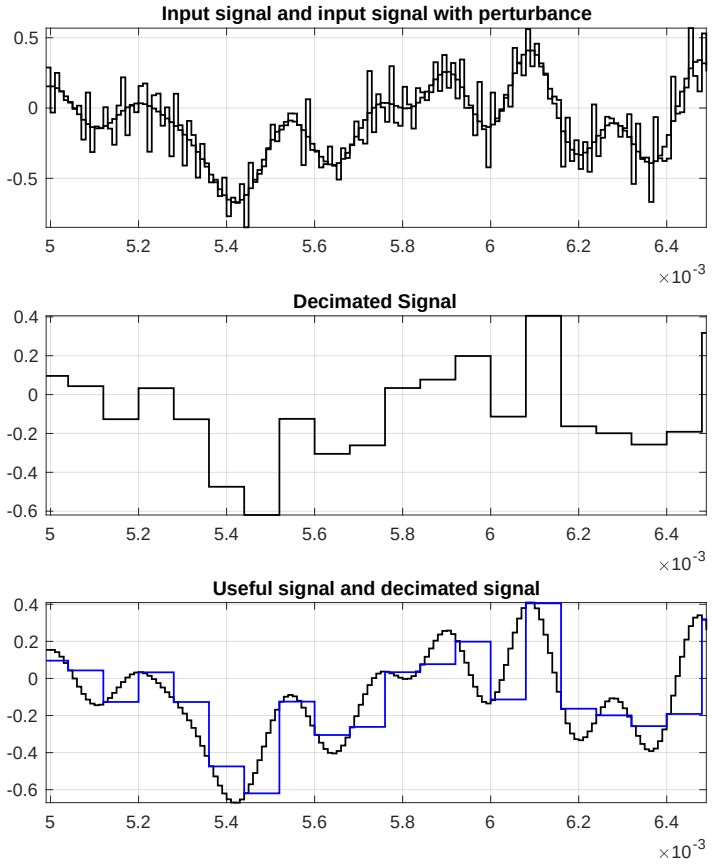


*Fig. 4: The results of the simulation (for pert = 0,2) (*decim_1.m, decim1.slx)

At the top you can see the useful (or desired) signal from the output of the `Discrete FIR Filter` block together with the same signal overlaid with the perturbation. The next signal represents the signal after the downsampling that represents the decimated signal.

At the very bottom the desired signal is displayed together with the decimated signal. This representation shows that the influence of the disturbance has been completely suppressed. At points at which the decimation values are taken, there is no difference to the sample values of the useful signal. Between the samples of the decimated signal there are M = 8 samples of the original useful signal.

The `Spectrum Analyzer` blocks are used to display the spectra at different

points in the model. These blocks can be connected to any other point. At the block `Spectrum Analyzer` the spectrum of the input signals is displayed, as shown in Fig. 5. The spectrum of the desired signal with a bandwidth of 6 kHz can be seen in the vicinity of frequency zero. Above the frequency of 20 kHz you can see the strong components of the perturbance for a factor `pert = 2`.

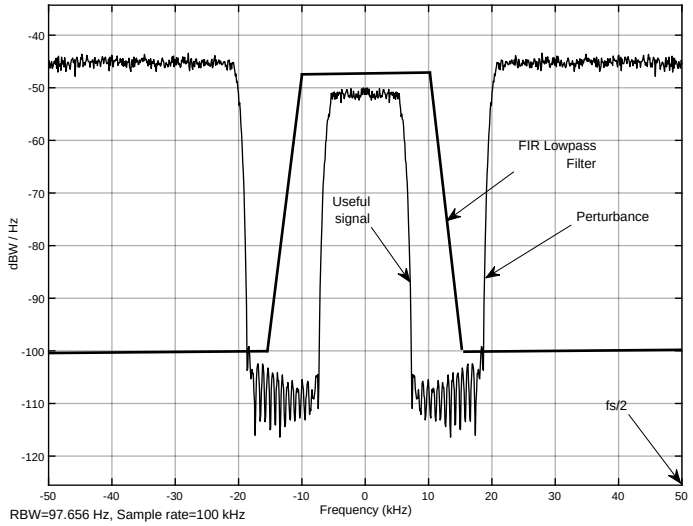


*Fig. 5: The spectrum of the input signals (for pert = 2) (*decim_1.m, decim1.slx)

The amplitude response of the decimation filter is also sketched in this illustration. Because the gap from 6 kHz to 20 kHz between the spectra of the desired signal and the perturbance is sufficiently large, the decimation filter is computed with the instruction:

```
hdec = fir1(nord2, 1.1/M); % FIR decimation filter
```

instead of with

```
hdec = fir1(nord2, 1/M); % FIR decimation filter
```

which corresponds to the theory.

With the blocks `Upsample`, `Gain` and `Spectrum Analyzer2`, the spectrum of the input signals can be displayed over four periods, as shown in Fig. 6.

The spectrum over several periods at the output of the FIR decimation filter is shown in Fig. 7. The remnants of the suppressed parts of the perturbance can be seen. After the downsampler the spectrum is shown in Fig. 8 and it corresponds to the representation from Fig. 2 c. This spectrum is recorded with the block `Spectrum Analyzer3`.

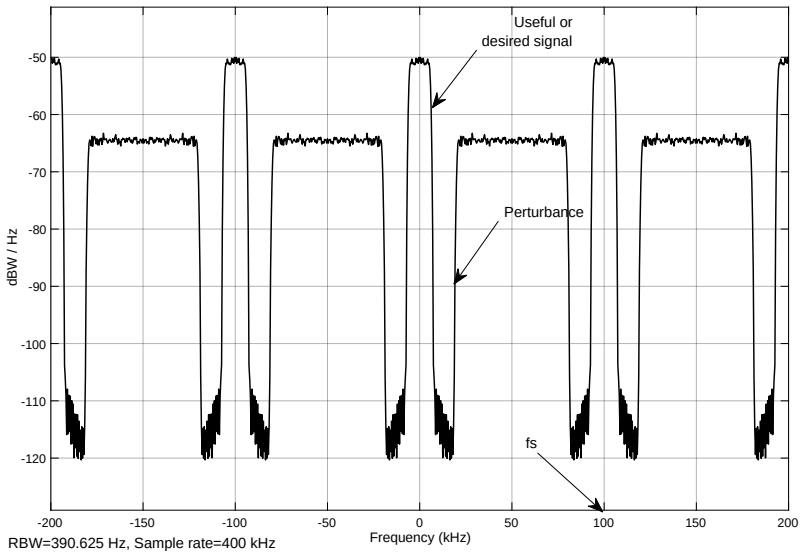


Fig. 6: The spectrum of the input signals over four periods (for pert = 0,2) (decim_1.m, decim1.slx)

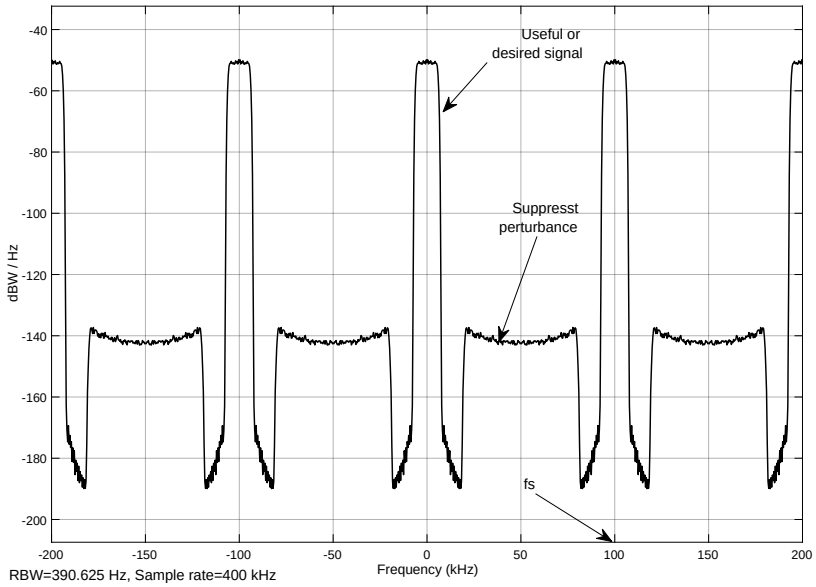


Fig. 7: The spectrum of the input signals after the FIR decimation filter (for pert = 0,2) (decim_1.m, decim1.slx)

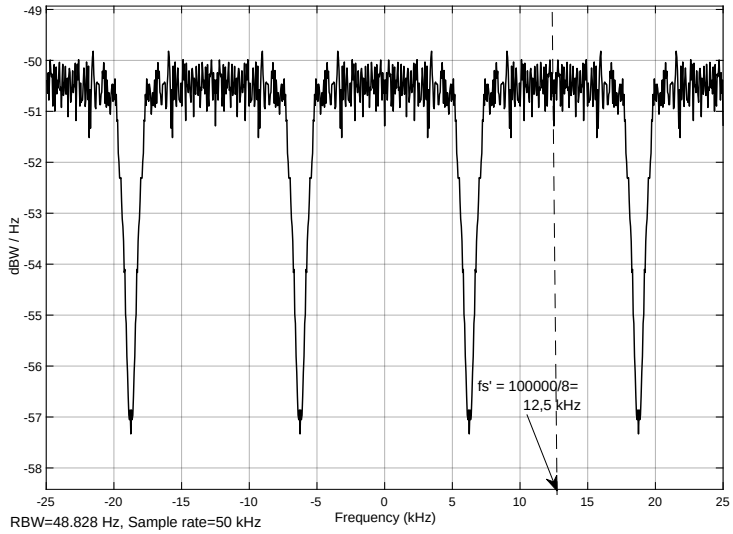


Fig. 8: The spectrum of the input signals after the downsampler (for pert = 2) (decim_1.m, decim1.slx)

2 Decimation with polyphase realization

The decimation with a polyphase implementation will now be described. This implementation is described first and then decimation is introduced. The polyphase filters allow first downsampling and then implementing the filters at the lower sampling frequency.

2.1 The polyphase realization of FIR filters

The polyphase realization of the FIR filter is a decomposition, which leads to a convertion into a parallel structure [1]. To illustrate this approach a causal FIR filter of order 8 with nine coefficients $H(z)$ is first coverted to a structure with two parallel filters.

$$\begin{aligned} H(z) =& h[0] + h[1]z^{-1} + h[2]z^{-2} + h[3]z^{-3} + h[4]z^{-4} + \\ & h[5]z^{-5} + h[6]z^{-6} + h[7]z^{-7} + h[8]z^{-8} \end{aligned} \tag{3}$$

As a sum of two filters, with one containing the even-indexed coefficients and the other containing the odd-indexed coefficients the following decomposition results:

$$\begin{aligned} H(z) =& (h[0] + h[2]z^{-2} + h[4]z^{-4} + h[6]z^{-6} + h[8]z^{-8}) + \\ & h[1]z^{-1} + h[3]z^{-3} + h[5]z^{-5} + h[7]z^{-7} = \\ & (h[0] + h[2]z^{-2} + h[4]z^{-4} + h[6]z^{-6} + h[8]z^{-8}) + \\ & z^{-1}(h[1] + h[3]z^{-2} + h[5]z^{-4} + h[7]z^{-6}) \end{aligned} \tag{4}$$

With the notations

$$\begin{aligned} E_0(z) &= h[0] + h[2]z^{-1} + h[4]z^{-2} + h[6]z^{-3} + h[8]z^{-4} \\ E_1(z) &= h[1] + h[3]z^{-1} + h[5]z^{-2} + h[7]z^{-3} \end{aligned} \tag{5}$$

the Eq. 4 now becomes:

$$H(z) = E_0(z^2) + z^{-1}E_1(z^2) \tag{6}$$

In a similar manner, by grouping the terms of Eq. 4 differently, you can get a different form:

$$H(z) = E_0(z^3) + z^{-1}E_1(z^3) + z^{-2}E_2(z^3) \tag{7}$$

Where now:

$$\begin{aligned} E_0(z) &= h[0] + h[3]z^{-1} + h[6]z^{-2} \\ E_1(z) &= h[1] + h[4]z^{-1} + h[7]z^{-2} \\ E_2(z) &= h[2] + h[5]z^{-1} + h[8]z^{-2} \end{aligned} \tag{8}$$

In the general case, an M-branch polyphase decomposition of an FIR filter of order N-1 with the transfer function

$$H(z) = \sum_{n=0}^{N-1} h[n]z^{-n} \tag{9}$$

result in

$$H(z) = \sum_{m=0}^{M-1} z^{-m}E_m(z^M), \tag{10}$$

where

$$E_m(z) = \sum_{n=0}^{\lceil N/M \rceil} h[Mn+m]z^{-n}, \qquad 0 \le m \le M-1 \tag{11}$$

Here is $\lceil N/M \rceil$ the integer part of N/M and $h[n] = 0$ for $n \ge N$. In Fig. 9 three different structures for a polyphase decomposition with M = 2, 3 and 4 branches are shown.

In Fig. 10 it is shown how the coefficients of an FIR filter are broken down into four (M = 4) polyphase filters. The coefficients of the FIR filter are shown at the top. The first filter of the z-transformation $E_0(z^4)$ contains every fourth coefficient of the FIR filter. The next polyphase filter $E_1(z^4)$ contains every fourth coefficient starting now at $n = 1$. This beginning means that the representation in Fig. 10c corresponds to the transfer function $z^{-1}E_1(z^4)$.

Similarly, the coefficients from Fig. 10d starting at $n = 2$ are the coefficients of the third polyphase filter $E_2(z^4)$. In the illustration, the transfer function corresponds thus to $z^{-2}E_2(z^4)$.

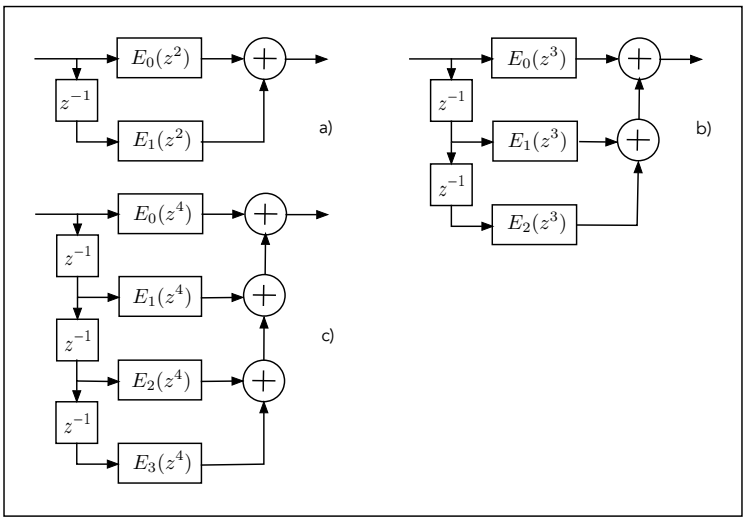


Fig. 9: Polyphase realization of an FIR transfer function

The representation from Fig. 10e with coefficients starting at $n = 3$ corresponds to the function $E_3(z^4)$ and the representation shows the transfer function $z^{-3}E_3(z^4)$.

When used in programs, the coefficients of the filters are combined in vectors. This means that e.g. the coefficients for $E_0(z^4)$ are:

```
E0z4 = [h[0],0,0,0,h[4],0,0,0,h[8],0,0,0,h[12],0,0,0,h[16], 0,0, ...]
```

Similarly, the coefficients for $E_1(z^4)$ are contained in the following vector:

```
E1z4 = [h[1],0,0,0,h[5],0,0,0,h[9],0,0,0,h[13],0,0,0,h[17], 0,0, ...]
```

The remaining two vectors are formed similarly:

```
E2z4 = [h[2],0,0,0,h[6],0,0,0,h[10],0,0,0,h[14],0,0,0,h[18], 0,0, ...]
E3z4 = [h[3],0,0,0,h[7],0,0,0,h[11],0,0,0,h[15],0,0,0,h[19], 0,0, ,,,]
```

Here `E0z4,E1z4, ...` are the vectors of the coefficients for the transfer functions $E_0(z^4), E_1(z^4), \ldots$.

The transfer functions $E_0(z), E_1(z), E_2(z), E_3(z)$, which contain the coefficients shown without the intermediate zero values, are also used later:

$$\begin{aligned} E_0(z) &= h[0] + h[4]z^{-1} + h[8]z^{-2} + h[12]z^{-3}... \\ E_1(z) &= h[1] + h[5]z^{-1} + h[9]z^{-2} + h[13]z^{-3}... \\ E_2(z) &= h[2] + h[6]z^{-1} + h[10]z^{-2} + h[14]z^{-3}... \\ E_3(z) &= h[3] + h[7]z^{-1} + h[11]z^{-2} + h[15]z^{-3}... \end{aligned} \tag{12}$$

These filters are actually viewed as polyphase decomposition filters.

It is now easy to imagine the transfer functions $E_m(z^M)$ or $E_m(z)$ with $m = 0, 1, 2, \ldots, M-1$ for the general case with an arbitrary value M and also the corresponding vectors.

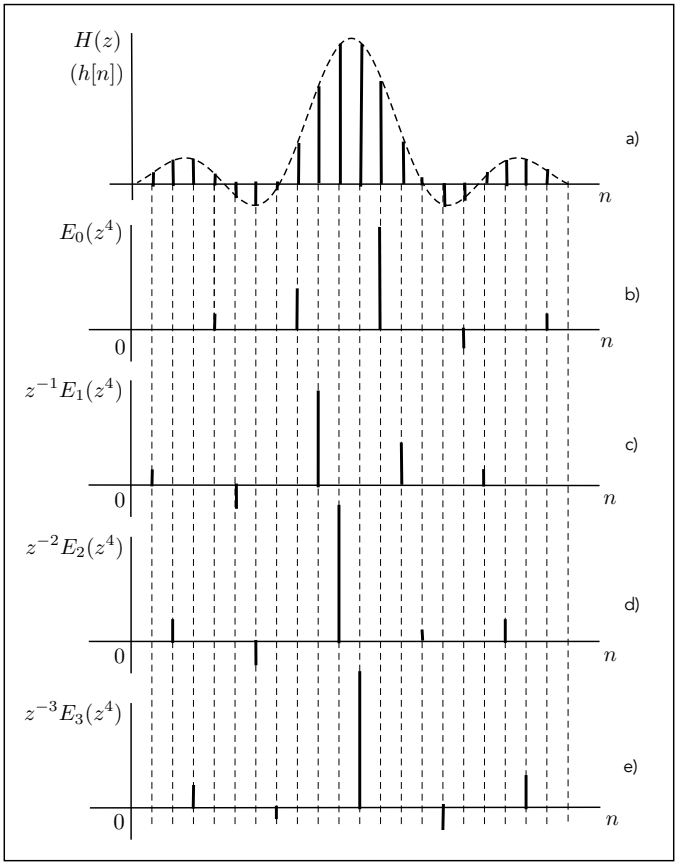


Fig. 10: Polyphase decomposition of an FIR impulse response into four branches

In the script `polyph_1.m`, which works with the Simulink model `polyph1.slx` a polyphase decomposition with M = 4 is examined. The following instructions store the transfer functions $E_m(z), m = 0, 1, 2, 3$ in the variables `Ez1(k, :), k = 1,2,3,4` as the lines of the matrix `Ez1`:

```
clear;
% -------- Nr. of branches
M = 4;
% -------- FIR Filter
ord = 20;              % Order of the filter
fr = 1/M;              % Relative frequency (MATLAB convention)
h = fir1(ord, fr);
% -------- Decomposition
h,
Ez1 = firpolyphase(h,M),   % The transfer functions Em(z)
```

From the values of the impulse response of the filter h

```
h =
```

```
Columns 1 through 8
  0.0025  0.0026  -0.0000  -0.0086  -0.0210  -0.0242  0.0000  0.0606
Columns 9 through 16
  0.1446  0.2191  0.2490  0.2191  0.1446  0.0606  0.0000  -0.0242
Columns 17 through 21
 -0.0210  -0.0086  -0.0000  0.0026  0.0025
```

one can understand the formation of the coefficients of the impulse responses of the transfer functions $E_m(z), m = 0, 1, 2, 3$:

```
Ez1 =
    0.0025   -0.0210    0.1446    0.1446   -0.0210    0.0025
    0.0026   -0.0242    0.2191    0.0606   -0.0086         0
   -0.0000    0.0000    0.2490    0.0000   -0.0000         0
   -0.0086    0.0606    0.2191   -0.0242    0.0026         0
```

The filters were determined with the MATLAB function `firpolyphase`. The

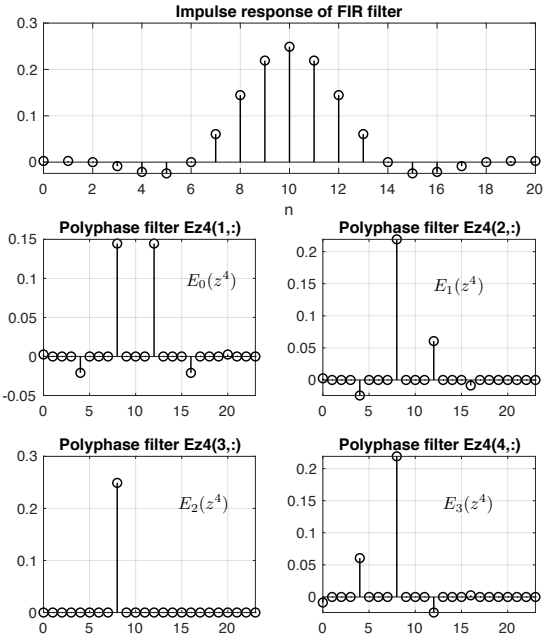


*Fig. 11: Polyphase decomposition of an FIR impulse response into four branches (*polyph_1.m, polyph1.slx)

coefficients of the impulse responses of the functions $E_m(z^4)$ are derived from the previous coefficients by adding M-1 = 3 zeros between these coefficients. This is done with the following instructions in the script:

```
% -------- The transfer functions Em(z^4)
nm = length(Ez1(1,:));
for k = 1:M
```

```
    Ez4_temp = [Ez1(k,:); zeros(M-1,nm)];
    Ez4(k,:) = reshape(Ez4_temp,1,M*nm),
end;

Ez4 =
  Columns 1 through 24
    0.0025   0   0   0  -0.0210   0    0    0   0.1446   0    0    0 ...
    0.0026   0   0   0  -0.0242   0    0    0   0.2191   0    0    0 ...
   -0.0000   0   0   0   0.0000   0    0    0   0.2490   0    0    0 ...
   -0.0086   0   0   0   0.0606   0    0    0   0.2191   0    0    0 ...
```

Each line of the matrix `Ez4` represents an impulse response of the transfer function $E_m(z^4), m = 0, 1, 2, 3$. The comparison of the matrix `Ez1` with the matrix `Ez4` shows the difference due M-1 = 3 zero values added.

Fig. 11 shows the decomposition of an impulse response of an FIR filter into M = 4 branches. It was also created in the script `polyph_1.m`.

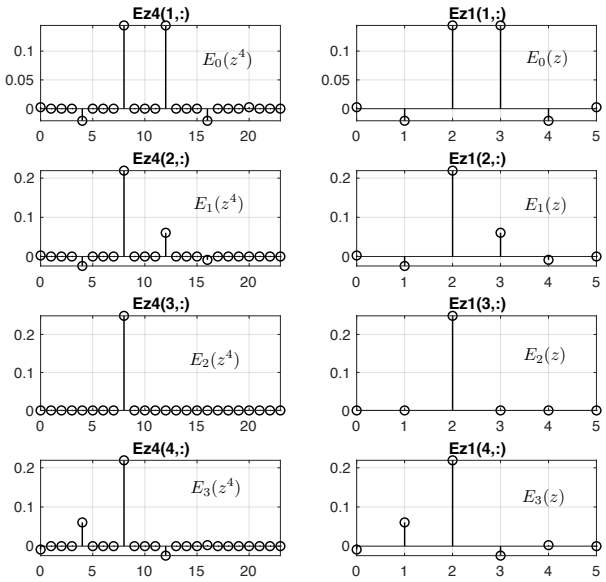


*Fig. 12: The difference between the functions $E_m(z^4)$ and the functions $E_m(z)$ (*polyph_1.m, polyph1.slx)

Fig. 12 shows the difference between the functions $E_m(z^4)$ and the functions $E_m(z)$. The indexes in MATLAB start with one, so the names of the vectors are a bit different from the names of the functions.

In the model `polyph1.slx` (Fig.13) the classic FIR filter is simulated above and the polyphase structure of the same filter for M = 4 is formed below it. The impulse responses of the polyphase filters are the rows of the matrix `Ez4` equal to `Ez4(1,:),Ez4(2,:),...,Ez4(4,:)` which coresponds to the functions

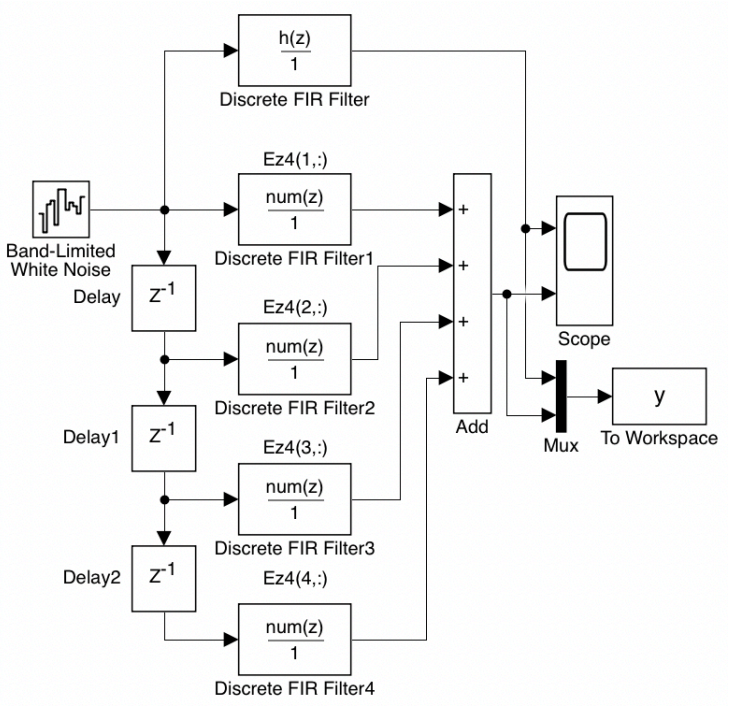


*Fig. 13: Simulink model with classic FIR filter and polyphase filter (*polyph_1.m, polyph1.slx)

$E_m(z^4), m = 0, 1, 2, 3$. On the block `Scope` you can see that the outputs of the classical and the polyphase filter are the same.

2.2 Decimation with filter in polyphase realization

In the area of multirate signal processing there are several equivalent structures [1]. One of these, which is important for the decimation, is shown in Fig. 14. In order to use this equivalence in a decimation with factor M, the FIR filter of the decimation must be decompose into M branches.

As shown, the FIR filter for decimation with factor M must be a Nyquist or M-band filter [1] whose impulse response has zero values at multiples of M.

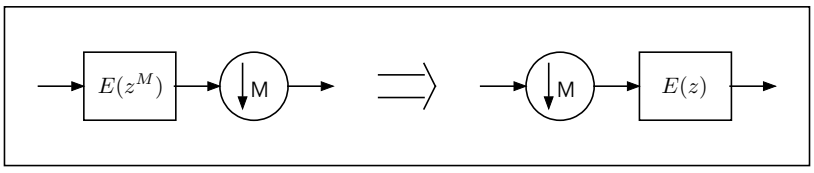


Fig. 14: Aquivalent structure in multirate signal processing

With the Simulink model `polyph2.slx` and script `polyph_2.m` a decimation

with factor M = 4, as in the previous simulation, is simulated without using the equivalency. The model is shown in Fig. 15. The downsample which should be connected after the sum of the outputs of the partial filters, has been moved in each branch which has the same effect. On the `Scope` block you can see that the two realizations of the decimation gives the same output signals

Now you can apply the equivalence and place the downsampling at the input of the filter, but the filters are no longer the functions $E_m(z^4), m = 0, 1, 2, 3$ or the vectors `Ez4 ( 1, :),Ez4 (2, :), ...,Ez4 (4,:)` but the functions $E_m(z), m = 0, 1, 2, 3$ or the vectors `Ez1(1, :),Ez1(2, :), ...,Ez1(4, :)`. This transformation of the decimation structure is examined with the script `polyph_3.m` and the Simulink model `polyph3.slx` for the same factor M = 4. The model is shown in Fig. 16. It is easy to imagine how the decimation with

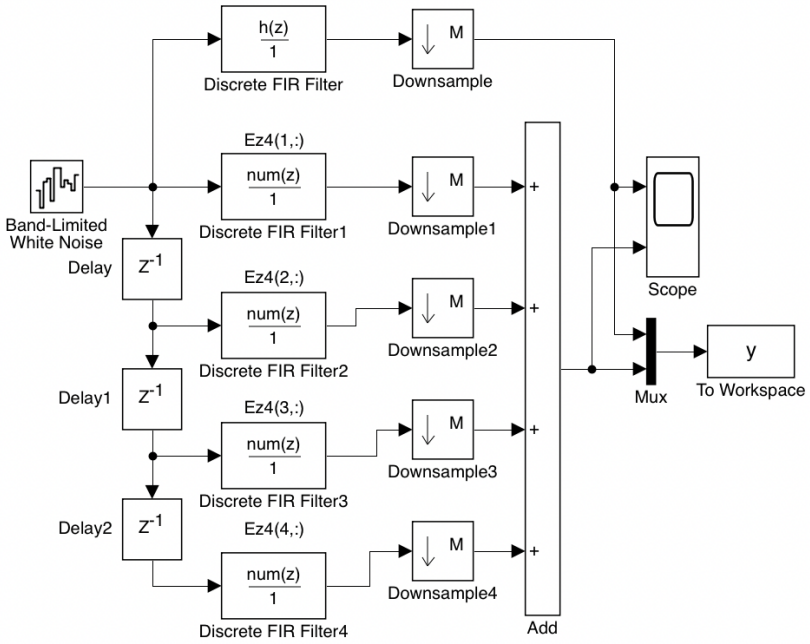


*Fig. 15: Simulink model of the decimation without using the equivalency (*polyph_2.m, polyph2.slx)

any factor M is to be designed in a polyphase implementation. There are several functions in MATLAB that use polyphase decomposition for decimation and other applications (such as interpolation). The simple function `firpolyphase` was used here.

The simulations for M = 4 are kept simple in design but can easily expanded by the readers. So you can easily expand the models for M = 8 and choose the FIR filter for certain input signals with a higher order. The spectra of the signals can also be followed by expanding the models with `Spectrum Analyzer` blocks.

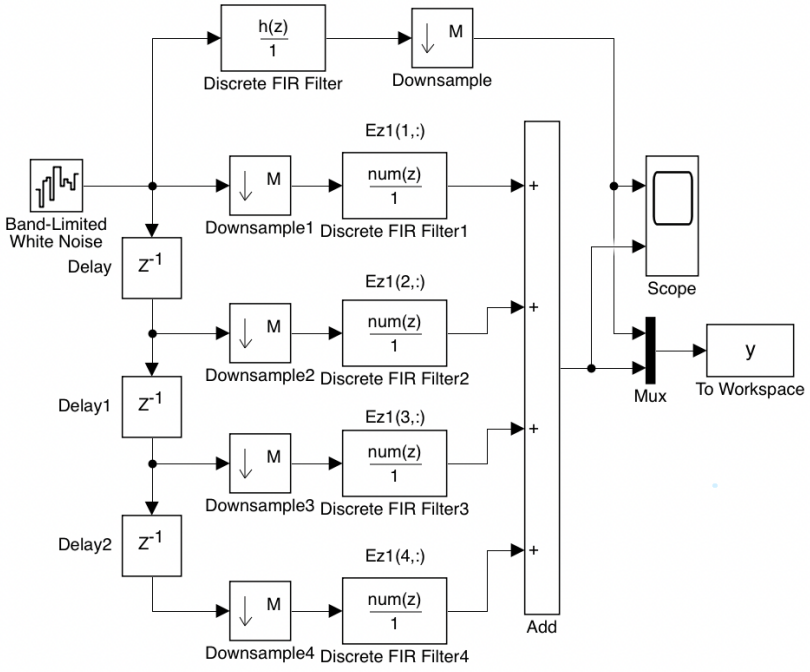


*Fig. 16: Simulink model of the decimation with the use of the equivalency (*polyph_3.m, polyph3.slx)

As in the simulation from Fig. 3, the input signals can be a useful or desired signal plus a disturbance and, depending on the gap between the useful signal and disturbance spectra, the order of the FIR filter can be selected through trials.

The polyphase implementation contains the same number of coefficients as the classic implementation and thus the same number of multiplications as an outlay. The main advantage is that the polyphase filters operate at the lower frequency after downsampling.

3 The interpolation process

3.1 Introduction

By interpolation, values are interpolated between the sampled values of a signal as if the original time-continuous signal had been sampled with a higher sampling frequency.

It is assumed that the sampled values of the time-continuous original signal before interpolation corresponded to the sampling theorem and thus contain all the information that enables interpolation. In concrete terms, this means that $f_s \geq 2f_{max}$. The sampling frequency of the continuous-time signal was designated with f_s and f_{max} is the maximum frequency in the spectrum of this

signal.

3.2 Interpolation of band limited lowpass signals

In Fig. 17a the structure of the classical interpolatin is shown. The input signal $x[nT_s]$, which fulfills the conditions shown above, is brought through upsampling to a frequency $f'_s = Lf_s$, where L is the interpolation factor (an integer). The upsampling adds L-1 zero values between the original sample values, so that the period T_s is divided into L equal intervals of the size $T'_s = T_s/L$. This gives the places where the FIR interpolation filter adds the interpolated values.

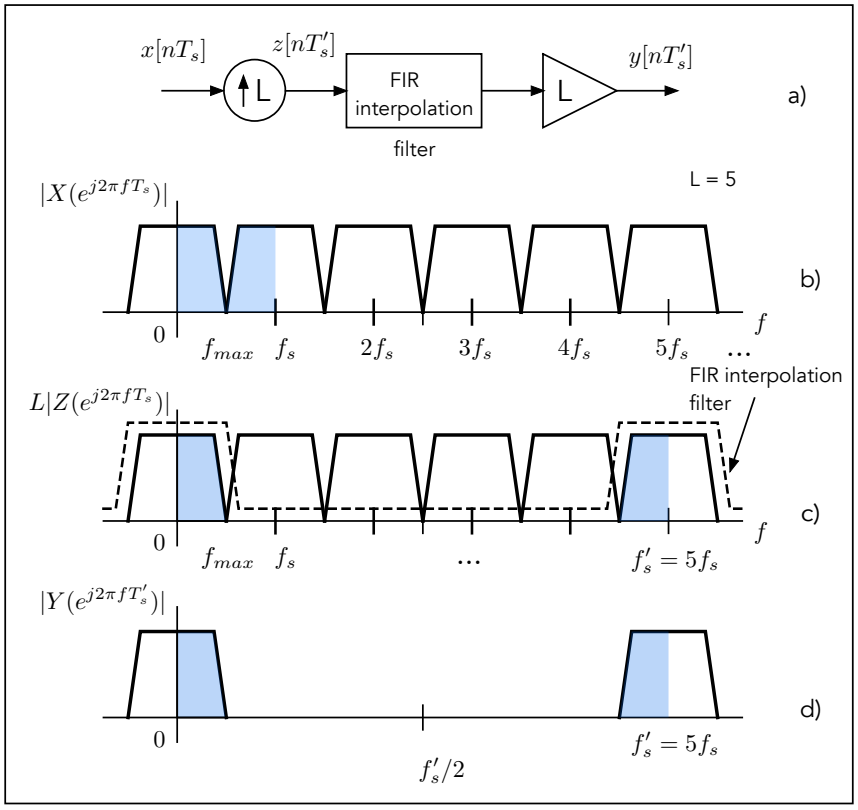


Fig. 17: The interpolation prozess (L=5)

In Fig. 17b the spectrum of the original signal is shown, it repeats itself periodically until $f \to \infty$. Assuming that L = 5, the figure shows the spectrum up to this frequency. The up-sampling now results in the signal $z[nT'_s]$ with a discrete sampling period $T'_s = T_s/L$ and a spectrum that is proportional to the spectrum of the signal $x[nT_s]$, as shown in Fig. 17c.

In order to obtain a signal with a spectrum as in Fig. 17d, which corresponds to the time-continuous original signal sampled with a sampling frequency f'_s, a

low-pass filter with an amplitude response that is sketched with a dashed line must be used.

If the discrete samples satisfy the sampling theorem, the continuous original signal can be perfectly reconstructed [1], [2]:

$$x(t) = \sum_{n=-\infty}^{\infty} x[nT_s] \frac{sin(\pi(t - nT_s)/T_s)}{\pi(t - nT_s)/T_s} \tag{13}$$

This formula shows how the continuous signal $x(t)$ can be determined from the sample values $x[nT_s]$. The formula equates a convolution of the sampled values as delta functions and an analog filter with the impulse response:

$$h(t) = \frac{sin(\pi t/T_s)}{\pi t/T_s} \tag{14}$$

It is an ideal analog filter of bandwidth $1/(2T_s)$, that can only be approximated in practice [1]. For a digital interpolation, the impulse response $h(t)$ must be digitized:

$$h[nT_s'] = \frac{sin(\pi(nT_s')/T_s)}{\pi(nT_s')/T_s} = \frac{sin(\pi n/L)}{\pi n/L} \quad \text{with} \quad -\infty < n < \infty \tag{15}$$

The ratio $T_s/T_s' = L$ was used here.

The FIR filter of the impulse response $h[nT_s']$ is also an ideal filter because it is infinitely extended and is not causal. For multiples of L ($n = kL, k \in \mathbb{Z}$), the values of the impulse response are equal to zero with the exception of $n = 0$, where the impulse response is equal to one.

To apply this filter, its extent must be limited e.g. from $n = -5L$ to $n = 5L$ and then shifted with $n = 5L$ so that it becomes causal. In digital signal processing, the extent is usually limited with a window function [1]. The simple limitation corresponds to a rectangular window function. The Hamming, Hann, Chebyshev filter window functions are very common window functions [1], [2].

In the script `sin_int_0.m` three filters are examined for the interpolation. The first filter `h` is the filter according to Eq. 15. The second `hw` is this filter weighted with the Hamming window function and the third `hnyq` is a Nyquist filter developed with the MATLAB function `fir1`, which by default also uses the Hamming window as a window function.

For this last filter you have to specify the pass frequency relative to half the sampling frequency f_s'. The Fourier-Transformation of the time-discrete impulse response according to Eq. 15 is an ideal rectangular low-pass function with a relative pass frequency $1/(2L)$. In the MATLAB convention, this means a pass frequency equal to $1/L$. This explains the call:

```
hnyq = fir1(ord,1/L);          % Nyquist filter
```

In Fig. 18 the impulse responses of the three filters for L = 5 are shown. The zero values for multiples of L can be clearly seen starting from n = 0 left and right. The impulse response of the first two filters have values equal to one at n

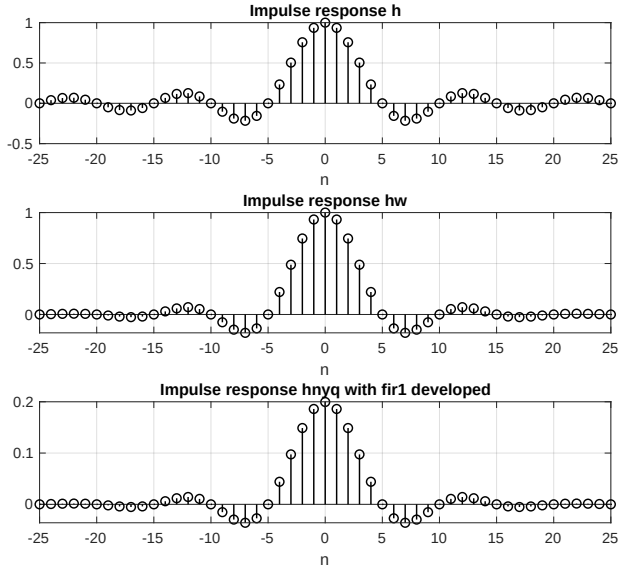


*Fig. 18: The impulse response of the three filters (*sin_int_0.m)

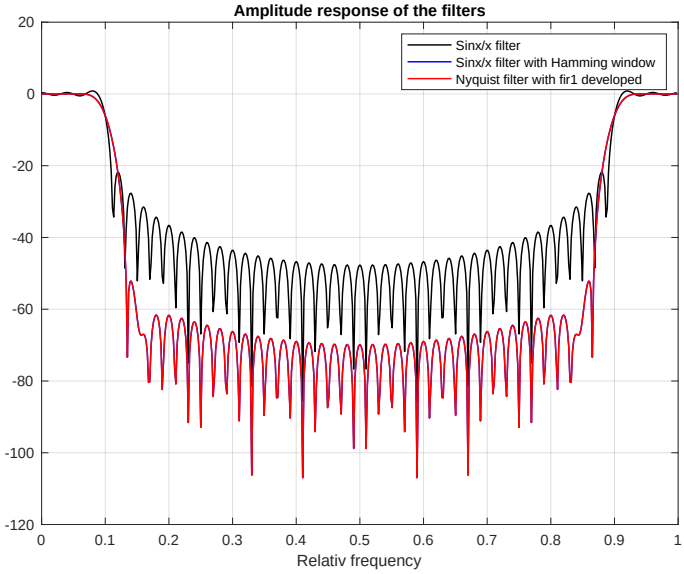


*Fig. 19: The amplitude response of the three filters (*sin_int_0.m)

= 0. This shows that the sample values of the signal to be interpolated appear unchanged in the interpolated values, which is certainly wanted.

The third filter has the value 0.2 at n = 0, which multiplied by L = 5 also results in one. The MATLAB function `fir1` normalizes the impulse response

so that the amplitude response in the pass band is equal to one. This explains why the structure of the interpolation according to Fig. 17a still contains the gain with L in the literature.

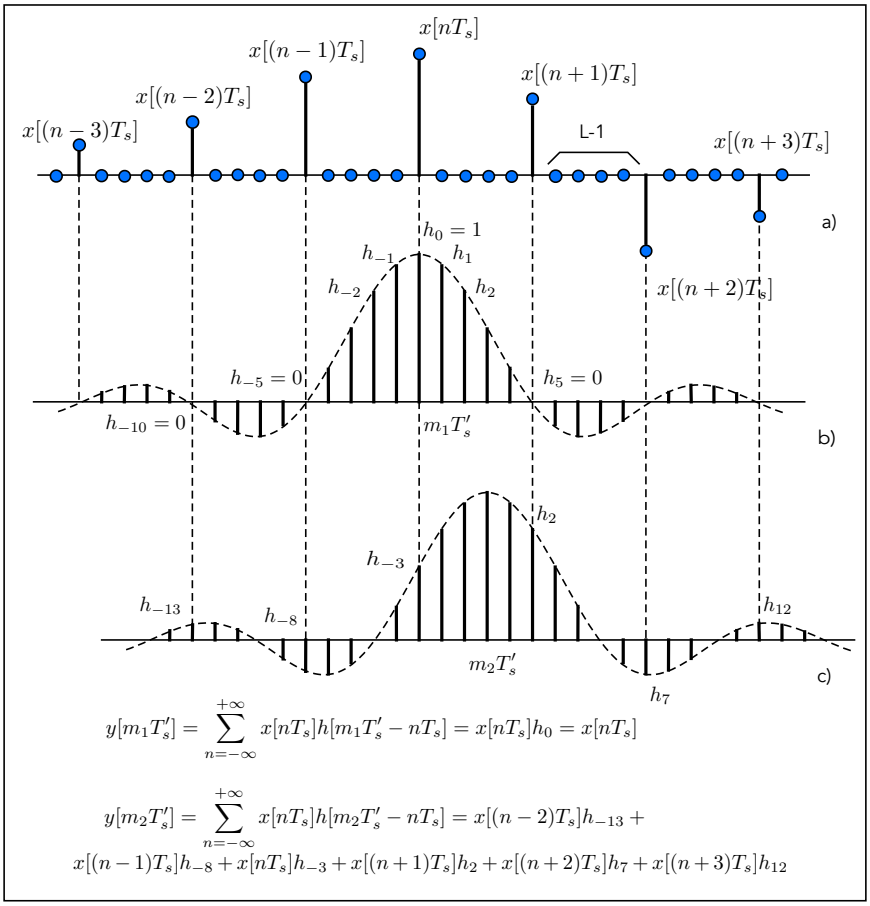


Fig. 20: The calculation of the convolution for two typical places of the impulse response

The amplitude responses of the three filters, which are normalized so that the pass band is equal to one, are shown in 19. The last two use the Hamming window and therefore the impulse responses and the amplitude responses are the same. The stop band begins at a frequency relative to the sampling frequency f'_s equal to $1/(2\mathrm{L}) = 0.1$.

With Fig. 20 the convolution with the non-causal impulse response of the filter is explained. The sequence $z[nT'_s]$ according to the structure from Fig. 17 a is shown at the top. You can see the L-1 intermediate values equal to zero, which define the places where the interpolated values are added by the filter.

In Fig. 20b is the position of the impulse response for the calculation of the

convolution shown, which leads to the output $y[m_1 T_s']$. You can see that only the sample value $x[nT_s]$ contributes to the output and results in $y[m_1 T_s'] = x[nT_s]$.

In contrast, Fig. 20 b shows a position for the impulse response for calculating the convolution that lies between the sample values. Here more samples contribute to the formation of the output $y[m_2 T_s']$.

With the causal impulse response, which arises from the non-causal one by shifting in the positive range, one only receives the response with this shift.

3.2.1 Simulation of an interpolation

In the script `sin_int_1.m` an interpolation with $L = 10$ is programmed in MATLAB. A band limited noise (<500 Hz) with a high sampling frequency (10000 Hz) is used as the input signal `xi`, which is last compared with the interpolated sequence.

```
% -------- Band limited noise as the initially signal
rng('default');
xn = randn(1,10000);
band = 400;                      % Bandwidth 400 Hz (Maximal 500 Hz)
hn = fir1(128,band*2/fsp);       % FIR filter
xi = filter(hn,1,xn);            % Original signal
```

The sequence to be interpolated `x` is obtained from this signal with a low sampling frequency equal to 1000 Hz.

```
x = xi(1:L:end);              % Sampled with fs
```

This signal is then interpolated with the factor $L = 10$ and compared with the original `xi`. The following instructions extend the sampled input sequence `x` with the $L - 1$ zero values:

```
% ----- Signal which will be interpolated with L-1 zero intermediate values
nx = length(x);
x = [x; zeros(L-1,nx)];
x = reshape(x,1,nx*L);
```

These instructions are used to determine the impulse responses of three FIR interpolation filters:

```
% -------- Interpolation Filter
ord = 10*L;       % Must be even number
n = -ord/2:ord/2;
h = sin(pi*n/L + eps)./(pi*n/L + eps);   % sinx/x impuls response
nh = length(h);
%w = hamming(nh);              % Hamming Window
w = hann(nh);                  % Hann window
hw = h.*w';                    % Sinx/x filter with window weighted
hnyq = fir1(ord,1/L);          % Nyquist filter
```

The interpolation is then carried out by:

```
% ----- Interpolation with the interpolation filter
%y = filter(h,1,x);
```

```
y = filter(hw,1,x);\input{kap_1.tex}

%y = filter(L*hnyq,1,x);
```

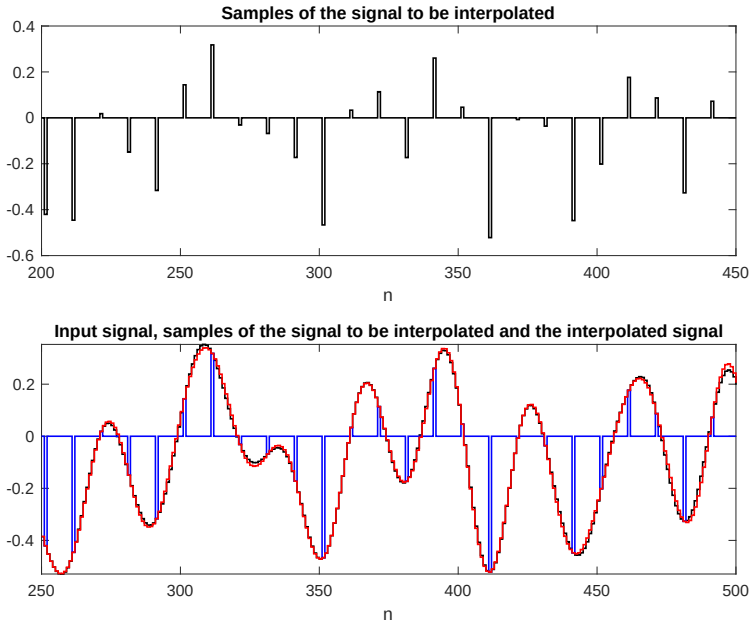


*Fig. 21: The results of the simulation (*sin_int_1.m)

The results of the simulation are shown in Fig. 21. At the top is the signal `x` with the $L - 1 = 9$ intermediate zero values. Below that the signal `x` is shown again together with the original `xi` and interpolated signal `y`. With the zoom function of the representations you can see the small differences between the original and interpolated signal.

With the script `sin_int_2.m` and model `sin_int2.slx` the interpolation from the previous example is simulated again. A Simulink model is very easy to understand and can easily be modified to creatively conduct further experiments. The model is shown in Fig. 22.

The band limited random input signal is generated with the blocks `Band-Limited White Noise` and `Discrete FIR Filter`. With a sampling frequency of 10,000 Hz and a band limitation up to 500 Hz, a very densely sampled signal is obtained with which the interpolated signal is compared.

The signal to be interpolated is obtained by sampling this signal at a sampling frequency of 1000 Hz using the `Zero-Order Hold` block. With the help of the block `Upsample` $L - 1 = 9$ intermediate zero values are added so that the sampling frequency is now 10000 Hz again. The interpolated values are generated with the interpolation filter from block `Discrete FIR Filter1`. The gain with factor L is necessary if the interpolation filter is normalized for an

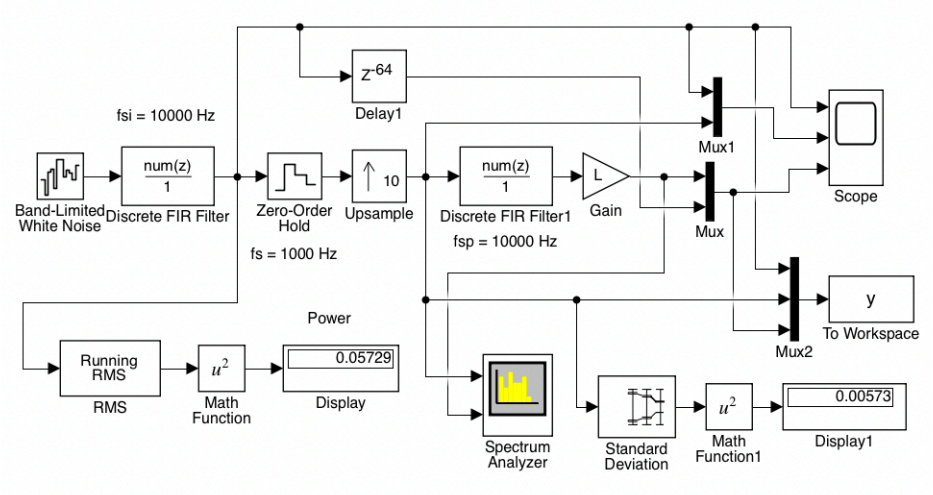


*Fig. 22: Simulink model of an interpolation (*sin_int_2.m, sin_int2.slx)

amplitude response one in the passband. This is the case when this filter is developed with the function `fir1`.

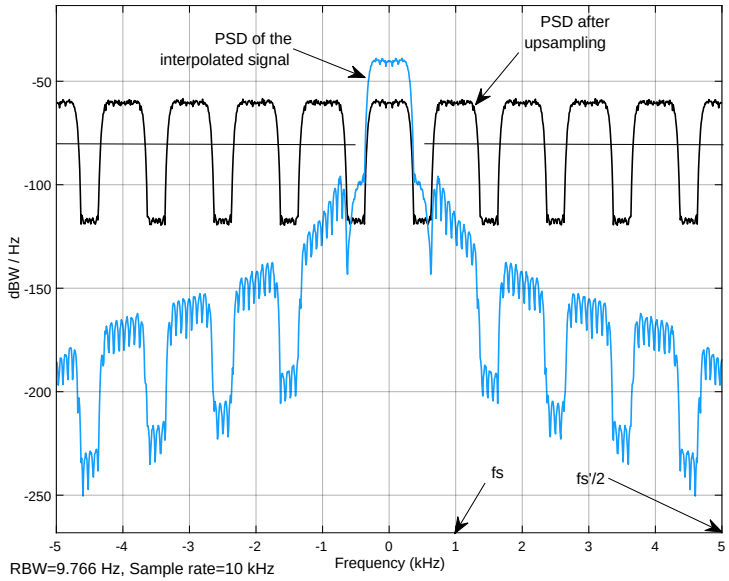


*Fig. 23: Power spectral density after upsampling and for the interpolated signal (*sin_int_2.m, sin_int2.slx)

The blocks `Display` and `Display1` are used to display the power at the points where the blocks `RMS` and `Standard Deviation` are connected. The mean power of the input signal after the `Discrete FIR Filter`, which depends on the band-

width of the random signal, is L times smaller after the block `Upsample` due to the L-1 intermediate zero values. With 300 Hz bandwidth, the average power of the original signal is 0.05729 Watts and after the upsampling it is 0.00573 Watts.

The mean power is restored with the gain L in the block `Gain`. At the block `Spectrum Analyzer`, the power spectral density after the upsampling (approx. -60 dBWatt/Hz) and the power spectral density of the reconstructed or interpolated signal (approx. -40 dBWatt/Hz) are displayed, as shown in Fig. 23.

The horizontal lines indicate the intermediate spectra of the upsampled signal. These are suppressed with the interpolation filter. The remnants of these intermediate spectra after the interpolation filter and amplifier can be seen in the second spectrum of this figure.

You can also connect the block `Spectrum Analyzer` to other places with the condition that both inputs have the same sampling frequency. If the sampling frequencies are different you have to use two blocks.

4 Interpolation with polyphase realization

An equivalent structure from the field of multi-rate signal processing, which is shown in Fig. 24, can be used advantageously for the interpolation.

The decomposition into polyphase filter structures was introduced in connection with decimation and is also valid for interpolation.

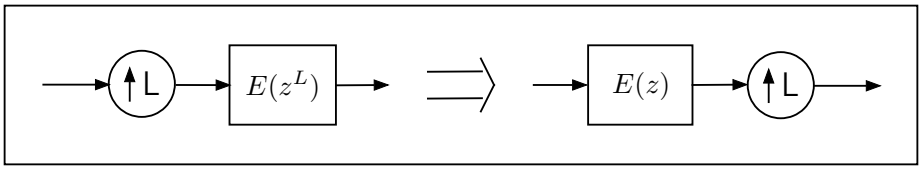


Fig. 24: Aquivalent structure in multirate signal processing

How one arrives an interpolation with a polyphase filter structure is shown with the help of a decomposition with L = 4. In Fig. 25a the classic form of interpolation with FIR filter of the transfer function $H(z)$ is shown. Below in Fig. 25b the structure is shown in which the interpolation filter is decomposed into polyphase filters:

$$H(z) = E_0(z^4) + z^{-1}E_1(z^4) + z^{-2}E_2(z^4) + z^{-3}E_3(z^4) \tag{16}$$

Here the transfer functions $E_m(z^4), m = 0, 1, 2, 3$ are defined in the equations 12. The delays can be shifted at the exits and the upsampling with factor L can be shifted in every path, as shown in Fig. 25c. Furthermore there is now the possibility to use the shown equivalents to get the structure from Fig. 25d. The blackened part corresponds to an unbuffer function, for which a block is available in Simulink that is used later in a simulation (see also Appendix). With the script `sin_int_3.m` and model `sin_int3.slx` an interpolation with L

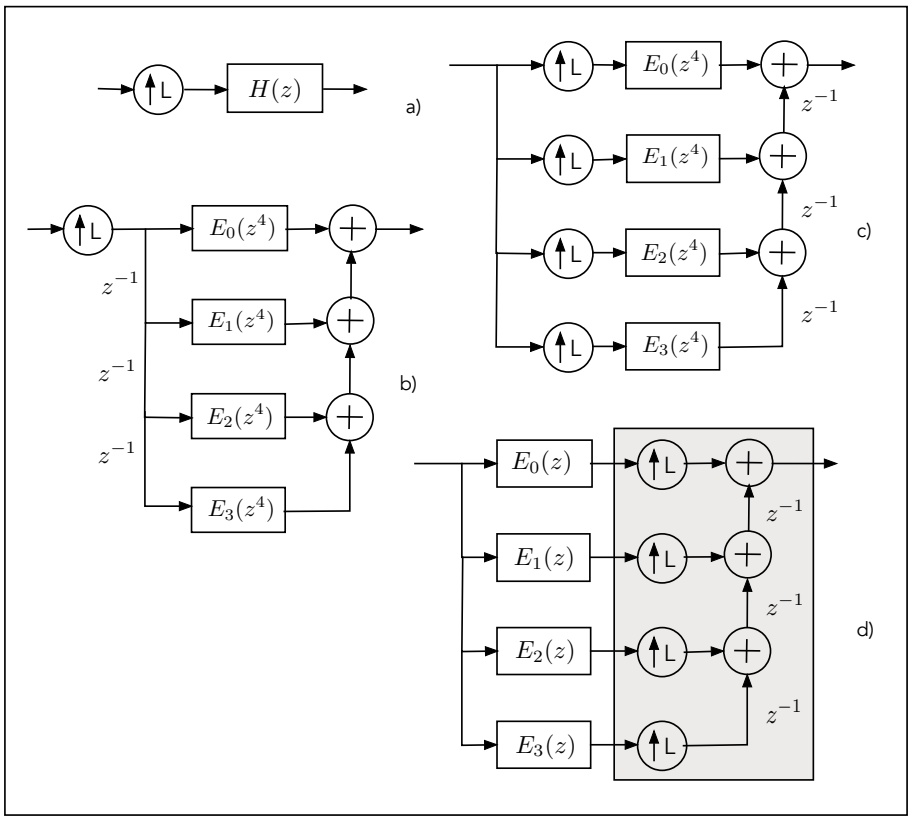


Fig. 25: a) Classic interpolation b) The interpolation filter with polyphase filters c) The delays placed to outputs and the upsampling shifted in the paths d) Interpolation with equivalent polyphase filters.

= 4 is simulated. The model is shown in Fig. 26. It corresponds to the structure from Fig. 25d.

With the blocks marked with shadow, the classic interpolation is formed. The solution with polyphase filters is shown below. The impulse responses of the FIR filters of this structure are determined with the following instructions in the script:

```
h = sin(pi*k/L + eps)./(pi*k/L + eps);   % sinx/x impuls response
h = h/L;
nh = length(h);
%w = hamming(nh);              % Hamming Window
w = hann(nh);                  % Hann window
hi = h.*w';                    % Sinx/x filter with window weighted
...
Ez1 = firpolyphase(hi,L);      % The polyphase filters
```

The impulse responses are stored in the lines of the matrix `Ez1`. The results of the simulation are shown in Fig. 27.

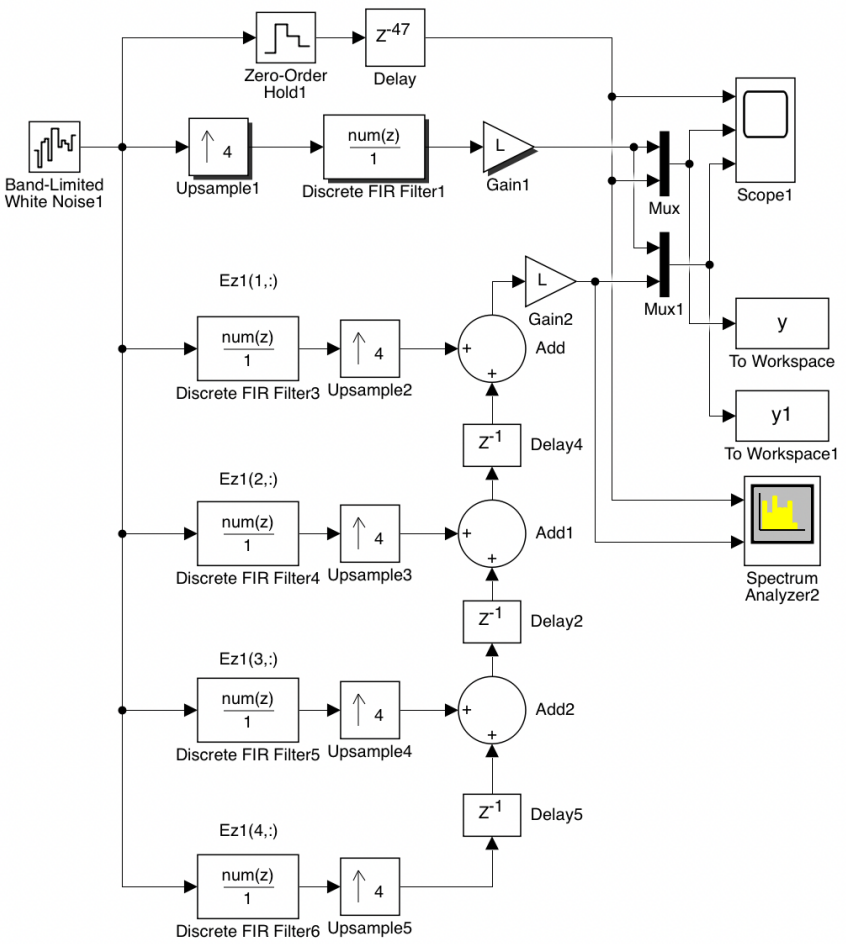


*Fig. 26: Simulink model for an interpolation with $L = 4$ (*sin_int_3.m, sin_int3.slx)

At the very top is the input signal that is to be interpolated. This signal is then shown together with the signal interpolated in the classic form. At the very bottom, the signal interpolated in classic form and the signal interpolated with polyphase filter are shown superimposed. You don't see any difference.

The number of multiplications is the same both in the classic type of interpolation and in the polyphase solution. The advantage of the latter is that the filters operate at the low sampling frequency.

With the script `sin_int_4.m` and model `sin_int4.slx` an interpolation with a factor of L=50 is simulated. The capability of the Simulink `Discrete FIR Filter` as SIMO filter (Single Input Multi Output) is used here. The model is shown in Fig. 28. The 50 polyphase filters as lines of the matrix `Ez1` are entered directly in the block `Discrete FIR Filter2`. Because

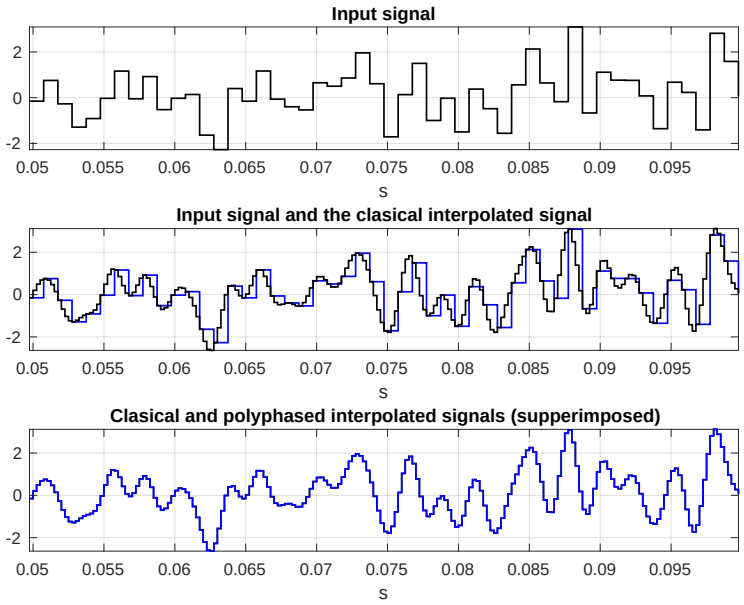


*Fig. 27: The results of the interpolation with L = 4 (*sin_int_3.m, sin_int3.slx)

the `Signal Dimensions` option is activated in the `DEBUG DIAGNOSTICS` menu of the model, the number of signals also appears at the output of this filter, in this case 50.

The composition of the 50 output signals of the polyphase sub-filters is here with the block `Unbuffer` obtained. The unbuffer function for L = 4 is shown in Fig. 25d in the blackened part.

Because the relative pass frequency of the interpolation filter is small 1/L = 0.02 (in the MATLAB convention), the order of this filter must be selected to be relatively high, e.g. 512. This means that the impulse response has slightly more than 10 intervals of the size L = 50. This impulse response is shown in the script and you can see that it contains the significant areas of the sinx/x function with infinite dimensions.

The amplitude response of the filter is also determined and displayed in the script and the pass frequency of 1/(2L) = 0.01 can be observed with the zoom function of the display.

The results of the simulation are shown in Fig. 29. At the top the signal to be interpolated is shown with a sampling frequency equal to 1000 Hz. Below that the same signal is shown together with the classically interpolated signal, now with 50000 Hz sampling frequency. Finally, the classically interpolated signal is shown superimposed together with the signal interpolated by the polyphase solution. There is no difference between these two signals.

In the script `sin_int_5.m` and model `sin_int5.slx` an interpolation with L = 50 with polyphase filters is simulated, whereby a band limited analog random signal is assumed. This signal is sampled and interpolated and is ultimately used

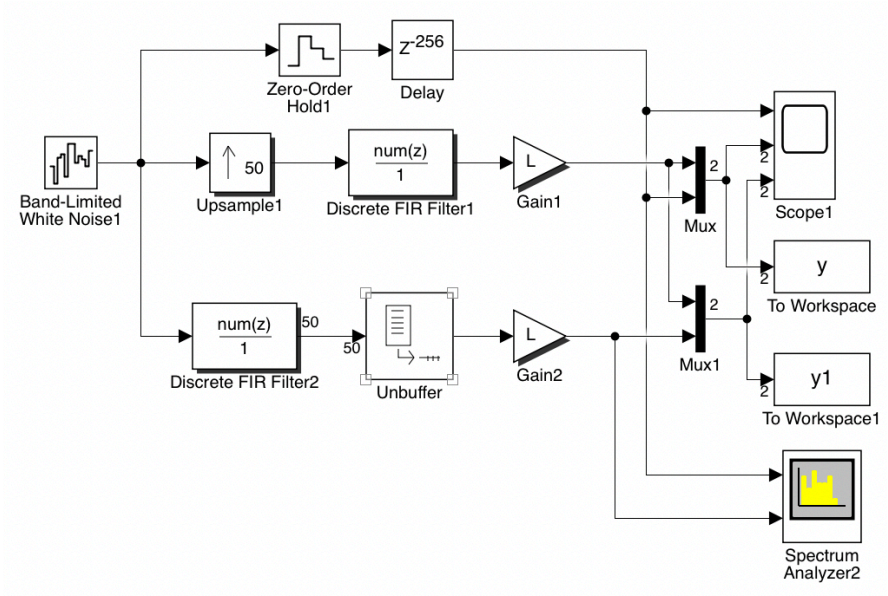


*Fig. 28: Simulink Modell of the interpolation with $L = 50$ (*sin_int_4.m, sin_int4.slx)

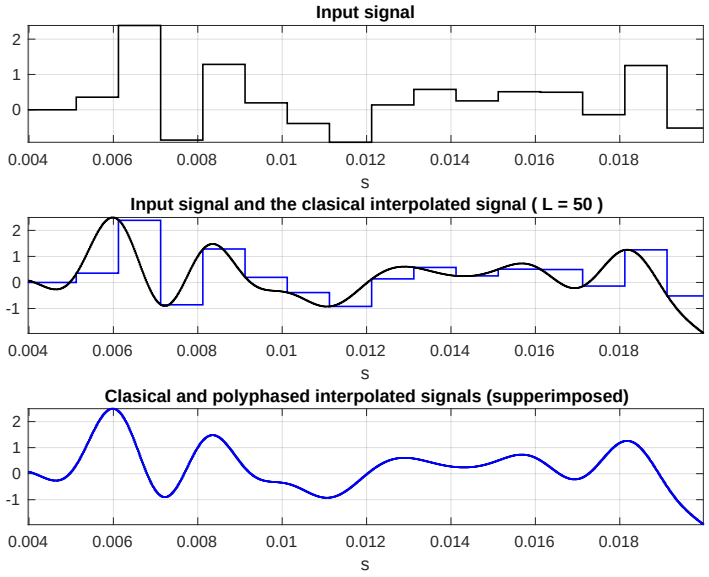


*Fig. 29: The results of the interpolation with $L = 50$ (*sin_int_4.m, sin_int4.slx)

for comparison with the interpolated signal. If the sampling theorem is violated, large differences arise between the original analog and the interpolated signal. Otherwise the interpolation gives a good reconstructed signal.

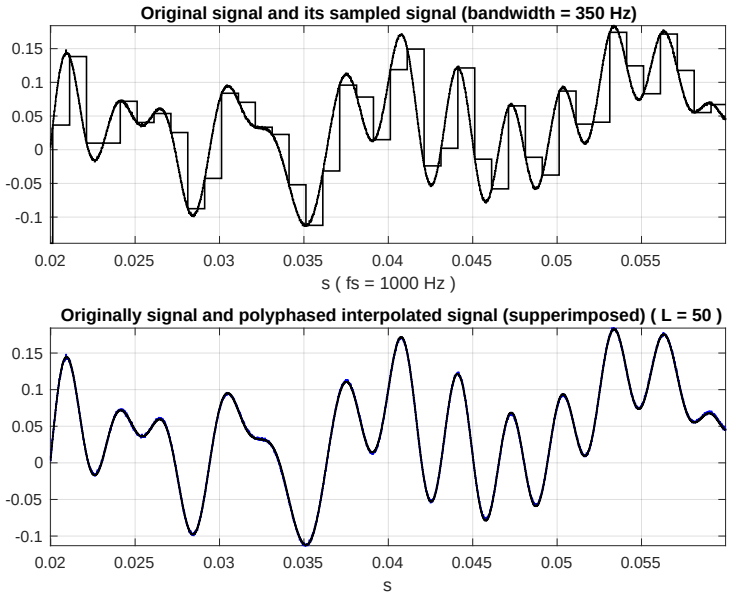


*Fig. 30: The results of the interpolation with $L = 50$ of a random band limited signal (*sin_int_5.m, sin_int5.slx)

The results of this simulation are shown in Fig. 30. At the top, the original analog signal is shown together with its sampled form. Below that, the same analog signal and the interpolated signal are shown superimposed. The bandwidth of the random signal with 350 Hz is smaller than half the sampling frequency of 1000 Hz and thus the sampling theorem is fulfilled.

The MATLAB scripts and the Simulink models are contained in the decim_interp_programs.zip file, which can be accessed via the following link: `https://dsprelated.com/blogimages/JosefHoffmann/decim_interp_programs.zip`

5 Appendix

5.1 Understanding the 'Unbuffer' function

In Fig. 31 it is shown how an 'Unbuffer' block for L = 4 works. The parallel occurring signals $z_1[nT_s], z_2[nT_s], z_3[nT_s], z_4[nT_s]$ are first extended by upsampling with L-1 zero values. This means that the sampling frequency is now $Lf_s = f_s'$. In the period T_s there are L intervals of the size $T_s' = T_s/L$.

The 4 signals occurring in parallel are serialized by delays $0, T_s', 2T_s', 3T_s'$. The illustration always shows one of the parallel signals together with the upsampled and delayed signal. The structure of the unbuffer block is shown again on the

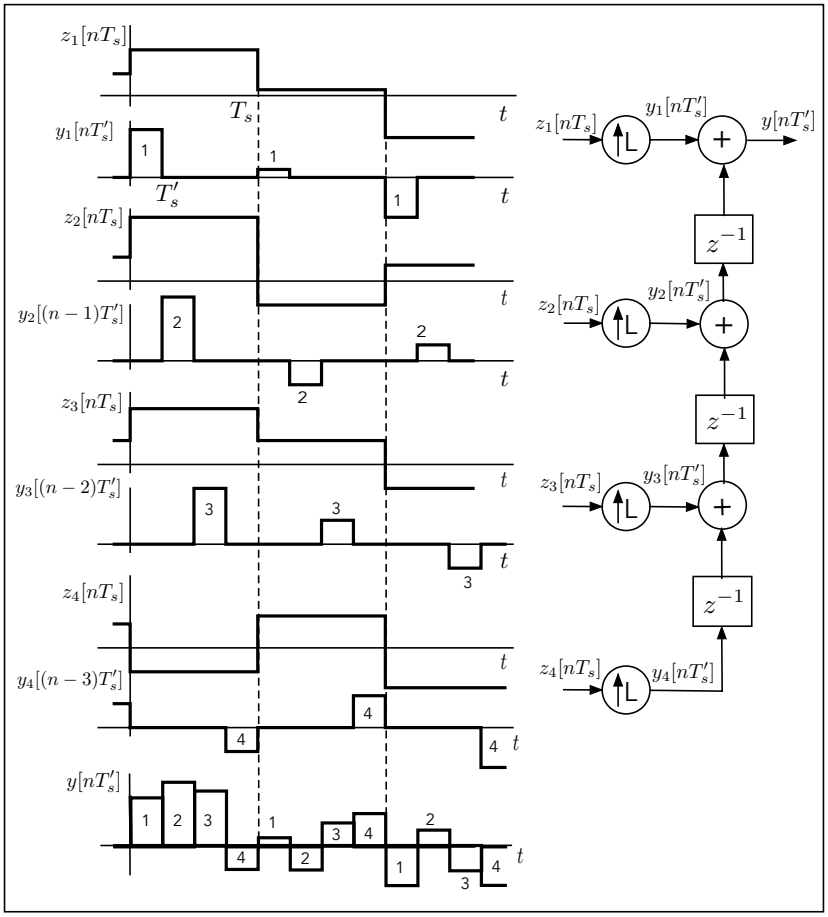


Fig. 31: The unbuffer function for L = 4

right in the picture. In the sum

$$y[nT_s'] = \sum_{m=1}^{4} y_m[(n-m+1)T_s'] \tag{17}$$

the parallel signals appear one after the other. To follow this better, the signals after the upsampling and delaying $y_1[nT_s'], y_2[(n-1)T_s'], y_3[(n-2)T_s'], y_4[(n-3)T_s']$ were marked with numbers.

References

[1] Mitra, Sanjit K.: *Digital Signal Processing. A Computer-Based Approach.* McGraw-Hill Publishing Company, 2005.

[2] Proakis, John G. and Manolakis, Dimitris G.: *Digital Signal Processing. Principles, Algorithms and Applications.* Prentice Hall, 2006.